

23

Object Detection

Finding and boxing many objects in one image

Not just what — but how many and where

AI/ML Engineer Learning Series · Deep Dive Edition

What it does	Finds objects AND draws a box around each one
Output	Boxes + labels + confidence for every object
Famous model	YOLO (You Only Look Once)
Beyond classification	Many objects, with their locations
Your roadmap	Object Detection with YOLOv8 (next project)

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is Object Detection?	Beyond one label
2	The Bounding Box	How a location is described
3	What the Model Outputs	Boxes, labels, confidence
4	IoU — Measuring Box Quality	How right is a box?
5	Two Families of Detectors	One-stage vs two-stage
6	YOLO — You Only Look Once	The popular fast detector
7	mAP — The Main Score	How detectors are graded
8	Using YOLO in Code	Surprisingly simple
9	Training on Your Own Data	The label format
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is Object Detection?

Image classification gives one label for a whole image. But what if there are several things in the picture? A street photo might have cars, people, signs, and bikes all at once. Object detection finds every object, tells you what each one is, AND draws a box around it showing where it is.

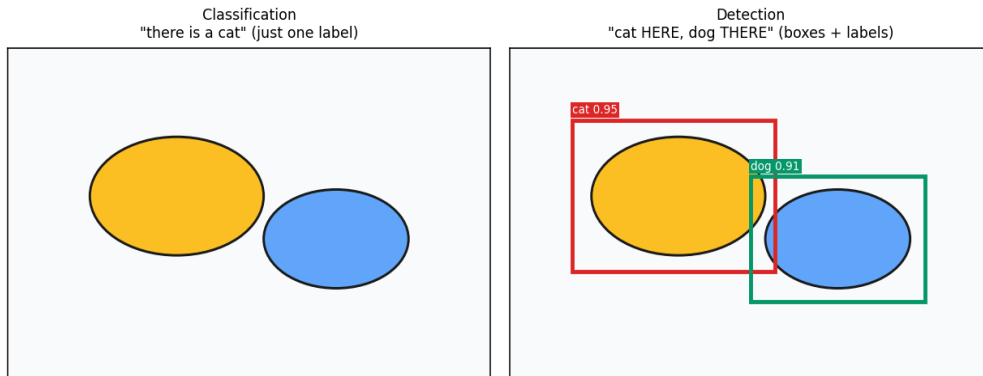


Fig 1 — Classification gives one label. Detection finds each object and boxes it with its own label.

So detection answers three questions at once: what objects are here, how many, and where exactly. This is much more useful for real scenes. It powers self-driving cars (spot every pedestrian and car), security cameras, retail shelf scanning, and quality control that needs to locate each defect, not just say one exists.

■ This is your next planned project: *Object Detection with YOLOv8*. This topic gives you the full mental model before you build it.

2. The Bounding Box

A bounding box is the rectangle drawn around an object. It's how the model says 'the object is right here'. A box is described by four numbers plus a label.

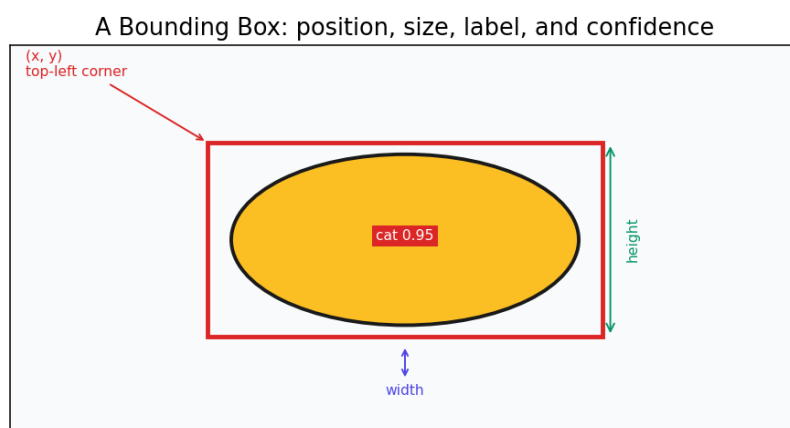


Fig 2 — A bounding box: where it starts (x, y), how big it is (width, height), plus a label and confidence.

Part	What it means
x, y	Position of the box (often the top-left corner)
width, height	How big the box is

label	What class the object is (cat, car, defect)
confidence	How sure the model is (0 to 1)

3. What the Model Outputs

For one image, a detector outputs a list of detections. Each detection is one box with its label and confidence. A photo with two cats and a dog gives three detections.

```
# What a detector returns for one image:  
[  
  {'box': [120, 80, 200, 240], 'label': 'cat', 'conf': 0.95},  
  {'box': [350, 100, 180, 220], 'label': 'cat', 'conf': 0.88},  
  {'box': [500, 150, 160, 200], 'label': 'dog', 'conf': 0.91},  
]  
# box = [x, y, width, height]
```

You then keep only the detections above a confidence threshold (say 0.5) to filter out weak guesses. One extra step, called Non-Max Suppression, removes duplicate boxes around the same object — the model often proposes several overlapping boxes, and this keeps just the best one.

■ *Non-Max Suppression (NMS) is a key clean-up step. Without it, you'd see several boxes stacked on the same object. NMS keeps the highest-confidence box and drops the overlapping copies. Most libraries do this for you automatically.*

4. IoU — Measuring Box Quality

How do we know if a predicted box is good? We compare it to the true box using IoU — Intersection over Union. It measures how much the two boxes overlap. IoU of 1.0 means a perfect match; 0 means no overlap at all.

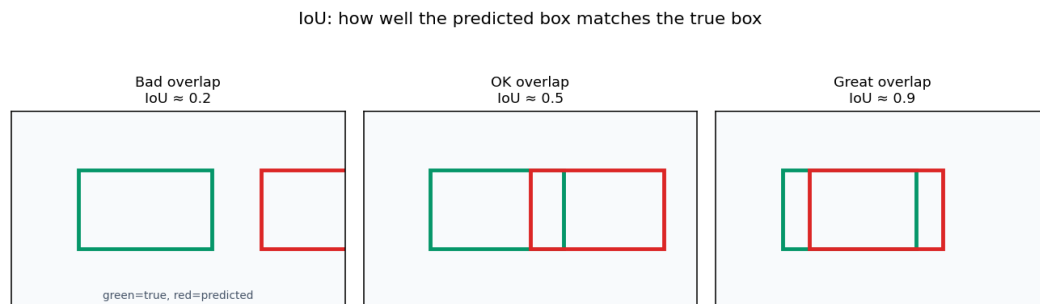


Fig 3 — IoU measures overlap between the predicted box (red) and the true box (green). Higher is better.

IoU is the overlapping area divided by the total combined area of the two boxes. A prediction usually counts as 'correct' if its IoU with a true box is above a threshold, commonly 0.5. This is how detection turns 'is the box in roughly the right place?' into a precise number.

5. Two Families of Detectors

Detection models come in two main styles, trading speed against accuracy.

Family	How it works	Trade-off
Two-stage	First find regions, then classify them	More accurate, slower
One-stage	Find and classify in one pass	Faster, great for real-time

Two-stage detectors (like Faster R-CNN) first propose areas that might contain objects, then look closely at each. They're accurate but slower. **One-stage** detectors (like YOLO and SSD) do everything in a single look at the image, making them fast enough for live video. For most projects today, especially anything real-time, one-stage detectors like YOLO are the go-to choice.

6. YOLO — You Only Look Once

YOLO is the most famous detector. The name says it all: it looks at the whole image just once and predicts all boxes in a single pass. That's what makes it fast enough for real-time video.

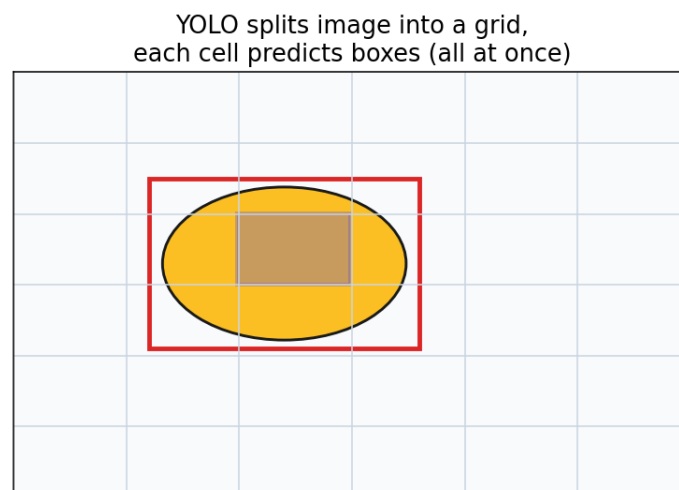


Fig 4 — YOLO splits the image into a grid. Each cell predicts boxes for objects centred in it — all at once.

YOLO divides the image into a grid. Each grid cell is responsible for predicting boxes for any object whose centre falls in that cell. Because every cell predicts at the same time, the whole image is processed in one go. The latest versions (YOLOv8 and newer) are extremely fast and accurate, and very easy to use — which is why YOLOv8 is on your project roadmap.

■ *YOLO has had many versions (v1 through v8 and beyond). They keep getting faster and more accurate. YOLOv8 from Ultralytics is a great modern choice — powerful, well-documented, and beginner-friendly.*

7. mAP — The Main Score

Detection isn't graded by simple accuracy, because a prediction must be both the right label AND the right location. The main score is mAP — mean Average Precision.

In plain terms, mAP combines two things: did the model find the objects (recall), and were its boxes correct and well-placed (precision, judged with IoU). It averages this across all classes into one number from 0 to 1 (or 0 to 100%). Higher mAP means a better detector. You'll often see scores like 'mAP@0.5', which means IoU threshold of 0.5 was used.

Term	Meaning
Precision	Of the boxes predicted, how many were correct
Recall	Of the real objects, how many were found
AP	Average Precision for one class
mAP	Mean of AP across all classes (the headline score)

8. Using YOLO in Code

Modern YOLO is remarkably easy to use. With the Ultralytics library, running detection on an image is just a few lines:

```
# pip install ultralytics
from ultralytics import YOLO

# Load a pre-trained YOLOv8 model
model = YOLO('yolov8n.pt') # 'n' = nano (small and fast)

# Detect objects in an image
results = model('street_photo.jpg')

# Show the image with boxes drawn
results[0].show()

# Get the detections
for box in results[0].boxes:
    cls = int(box.cls) # class id
    conf = float(box.conf) # confidence
    xxyy = box.xyxy[0].tolist() # box coordinates
    print(model.names[cls], conf, xxyy)
```

9. Training on Your Own Data

To detect your own objects (like specific product defects), you train YOLO on labelled images. Labelling means drawing a box around each object and saving its class and position. Tools like Roboflow or Labellmg help you do this.

YOLO uses a simple label format: one text file per image, with one line per object giving the class and the box.

```
# For each image, a matching .txt label file:
# class_id x_center y_center width height
# (all numbers are fractions of image size, 0 to 1)

0 0.45 0.55 0.30 0.40 # object of class 0
1 0.70 0.42 0.18 0.25 # object of class 1

# Then train with one command:
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.train(data='mydata.yaml', epochs=50, imgsz=640)
```

■ The data.yaml file just lists where your train/val images are and the names of your classes. Roboflow can export your labelled dataset already in this exact YOLO format, which saves a lot of setup.

10. Common Mistakes

Mistake	Why it's a problem	Fix
Confusing it with classification	Detection needs locations too	Output boxes, not just labels
Ignoring NMS	Duplicate boxes on one object	Use Non-Max Suppression
Bad or sloppy labels	Model learns wrong boxes	Label carefully and consistently
Too high confidence threshold	Misses real objects	Lower the threshold
Too few training images	Poor detection	Get more labelled data + augment
Judging by accuracy only	Misses location quality	Use mAP and IoU

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Object detection	find + box + label every object
Bounding box	x, y, width, height around an object
Confidence	how sure the model is (0-1)

IoU	overlap between predicted and true box
NMS	removes duplicate boxes on one object
One-stage (YOLO)	fast, real-time detection
Two-stage (R-CNN)	slower, very accurate
mAP	main detection score (higher = better)
Load YOLO	YOLO('yolov8n.pt')
Detect	results = model('image.jpg')
Show result	results[0].show()
Train YOLO	model.train(data='d.yaml', epochs=50)
Label format	class x_center y_center w h (0-1)

The one idea to remember: object detection finds every object in an image and boxes it with a label and confidence. YOLO does this fast in one pass. Quality is judged by IoU (box overlap) and mAP (the overall score).

Next Topic: OpenCV — the essential toolkit for working with images and video in code: loading, resizing, drawing boxes, and feeding frames to your detection models.